Masking-Agnostic Deep Learning for Side-Channel Analysis via Learnable Bilinear Combining

Tom Soustiel, Moshe Avital, and Nurit Spingarn

Technion – Israel Institute of Technology, Haifa, Israel
tom.soustiel@campus.technion.ac.il

Abstract. Cryptographic systems, while mathematically secure, remain vulnerable to Side-Channel Analysis (SCA) attacks that exploit physical leakages such as power consumption and electromagnetic emissions. Deep-learning-based SCA on first-order masked AES has been demonstrated black-box—without invasive mask probing and without hard-coded knowledge of the masking scheme—by Perin et al. (TCHES 2022), but the result is obtained through a search of 500 generic MLP/CNN configurations over a 10^9-option hyperparameter space, providing no analytical handle on *why* the resulting networks succeed.

This paper presents a masking-agnostic black-box architecture that reaches the same regime through an explicit algebraic prior rather than undirected search. We introduce a learnable *bilinear combiner* based on Canonical Polyadic (CP) decomposition that replaces hand-coded demasking layers, allowing the model to discover the demasking operation directly from data with only unmasked SBox-output supervision. We identify and resolve a previously unreported *gradient dead zone* that traps bilinear combiners applied to softmax probabilities, and show that operating on raw logits with LayerNormalization, a skip connection, L2 regularisation, and Gaussian noise injection yields stable convergence. The construction generalises naturally to higher-order masking: a d-share scheme corresponds to a d-linear interaction and a $(d{+}1)$-dimensional CP decomposition with d component branches.

On the ASCAD_r benchmark, the architecture is selected through a 4-phase manual ablation (rather than hyperparameter search) and recovers all 14 masked key bytes with GE $= 2^0$ in 2–4 attack traces—within the trace range of the best published black-box result—and improves clearly over the XOR-supervised multi-task baseline of Marquet and Oswald (COSADE 2024), which requires 3–14 traces despite being told that the masking scheme is XOR. The contribution is representational: a structured, interpretable architecture whose inductive bias matches the algebraic class of demasking and extends cleanly to higher-order masking, obtained without large hyperparameter searches.

Keywords: Side-Channel Analysis · Deep Learning · Multi-Task Learning · Masking Countermeasures · Tensor Decomposition · AES

1 Introduction

Cryptographic systems, although mathematically secure, are exposed to information leakage through side channels. Statistical tools currently exist that enable extraction of sensitive information from these systems [10,3,16]. Deep learning has significantly expanded the capabilities of side-channel attacks beyond traditional methods such as Random Forests and SVM, which were limited by high computational costs and large sample requirements [15,17].

However, current deep learning approaches to SCA against masked implementations impose a fundamental trade-off on the attacker:

1. **Mask-label dependency:** Some approaches require labeled internal masking data—the random mask values used during encryption—for training. Obtaining these labels in practice demands invasive or semi-invasive probing of the target chip. Recent work has demonstrated that optical techniques such as Laser Voltage Probing can extract individual register states—including mask shares—from the chip backside [12,13], but this requires specialized failure-analysis equipment (laser scanning microscopes costing hundreds of thousands of dollars), chip depackaging, and circuit reverse engineering to locate the mask generation logic [14,8]—a costly process that itself constitutes a separate research effort. Transformer-based approaches such as GPAM [4] achieve impressive results but operate in this white-box setting, requiring access to masking labels and internal values.
2. **Architectural-knowledge dependency:** Other approaches avoid mask labels but instead embed hardcoded algebraic knowledge of the masking scheme into the model architecture. Marquet and Oswald [17] use a dedicated XorLayer that implements the exact XOR probability convolution, requiring *a priori* knowledge that the masking scheme is boolean XOR. The attacker must know (or correctly guess) the masking scheme before designing the model.

It is important to note that these two dependencies are not truly orthogonal. Designating a value as a "mask" presupposes that the attacker already knows the masking scheme in use—which intermediates carry masks, how many shares, when masks refresh. Item 1 therefore silently inherits item 2: mask-label dependency is a strict superset of architectural-knowledge dependency, and any approach paying the (very high) cost of obtaining mask labels has *also* paid the architectural-knowledge cost as a prerequisite. A black-box attacker, by contrast, pays neither.

Neither side of this trade-off is realistic in a true black-box attack scenario, where the attacker has access only to power or EM traces and the corresponding plaintexts, with no knowledge of the implementation's internal structure. Recent work has explored alternative supervision regimes from orthogonal angles: weakly-profiling settings that reduce reliance on labeled data [27], and non-profiled deep-learning collision attacks that exploit cipher structure to bypass mask labels entirely [24]. These approaches share our motivation of reducing supervision requirements but do not address the architectural-knowledge dependency on the masking scheme itself.

The goal of this work is the development of a deep learning-based attack model operating as a "Black Box"—without any prior knowledge about the implementation method, and more importantly, without relying on internal intermediate data of AES encryption, such as internal masking data or intermediate value labels.

Contributions. The viability of black-box DL-SCA on masked AES has been established by Perin et al. [20] via large random hyperparameter search. Our contribution is to show that the same regime can be reached through an explicit algebraic prior, and to expose what that prior is. Specifically:

- **An explicit algebraic prior for demasking.** We propose a **BilinearCombinerLayer** based on CP tensor decomposition that replaces hand-coded demasking layers. The layer learns the demasking operation from data via gradient descent, requiring only unmasked SBox-output labels for supervision—no mask labels, no intermediate masked values, no hardcoded XOR. Unlike a generic deep network, the architecture states what algebraic class of operation it is searching over.
- **The gradient dead zone.** We identify and resolve a previously unreported failure mode: applying bilinear operations to softmax probability distributions produces $O(1/256^2)$ gradient magnitudes, trapping the model at the uniform distribution indefinitely. Operating on raw logits with LayerNormalization restores $O(1)$ gradient flow.
- **A skip-plus-regularisation training recipe.** A skip connection bootstraps backbone learning before the combiner converges, while L2 regularisation and Gaussian noise injection then force the model to exploit the bilinear combiner rather than overfit through the skip path.
- **Higher-order extension via CP decomposition.** The two-component bilinear construction corresponds to first-order (d=2) masking; a d-share scheme is a d-linear interaction representable by a (d+1)-dimensional CP decomposition with d component branches. This gives a clean, structured route to higher-order masking that a generic MLP/CNN architecture does not offer.
- **A 4-phase ablation as an alternative to hyperparameter search.** We arrive at the final architecture through four explicit phases, each addressing a specific failure mode—direct prediction, MLP combiner, bilinear-on-softmax (dead zone), and bilinear-on-logits with regularisation. This replaces an $O(10^3)$-trial random search with a handful of architectural decisions, each accompanied by a mechanistic explanation.
- **Quantitative evaluation on ASCAD_r.** Our model recovers all 14 masked key bytes with GE = 2^0 in 2–4 attack traces, within the trace range of the strongest published black-box result [20], and clearly better than the XOR-supervised multi-task baseline of Marquet and Oswald [17], which requires 3–14 traces despite hardcoding the XOR scheme.
- **An SNR analysis of ASCAD_r** establishing that per-byte difficulty variations arise from the physical leakage profile and persist across architectures, suggesting that the remaining bottleneck is the data, not the model.

2 Background

2.1 AES Encryption

The Advanced Encryption Standard (AES) [19] is a symmetric block cipher that operates on input of 16 bytes (128-bit plaintext). The encryption process consists of 10 rounds (for a 128-bit key), each round including four operations on the State Matrix (a 4×4 matrix of bytes): (1) SubBytes (SBox)—non-linear substitution of each byte; (2) ShiftRows—circular shifting of the matrix rows; (3) MixColumns—linear mixing of the matrix columns; (4) AddRoundKey—XOR addition of the state matrix with a round-specific sub-key.

2.2 SBox as the Attack Target

The SBox operation in the first encryption round is the classical vulnerability point for SCA:

$$s_1 = \texttt{Sbox}[p \oplus k], \tag{1}$$

where p is the plaintext byte and k is the secret key byte. This equation directly links known information (plaintext), a part of the secret (key byte), and an intermediate result.

The SBox is the only non-linear function in AES, based on inversion in the Galois Field GF(2^8). A change of a single bit in the input leads to significant and unpredictable changes across many output bits, creating a physical signature. From a hardware perspective, when the microcontroller computes the SBox output and writes it to a register or data bus, this operation consumes electrical power that is directly dependent on the data being processed—each bit that changes state causes charging or discharging of parasitic capacitance, producing a current spike proportional to the Hamming Weight [16].

2.3 Masking Countermeasures

In a standard (unprotected) AES implementation, the intermediate values are processed directly, and a single time point in the power trace may leak sensitive information (first-order leakage).

In an implementation with boolean masking (which we focus on in this work), a random mask r is generated, and instead of working directly on s_1, the system computes:

$$z = s_1 \oplus r. \tag{2}$$

Each sample in the trace on its own appears random, since the mask is random and the attacker has no direct access to it.

To attack a masked implementation, the attacker needs to combine two time points in the trace—one containing leakage about the mask r, and the other about the masked value z. Only the joint distribution of these two points depends on the secret value s_1—this constitutes second-order information leakage [23].

Many researchers use the secret mask values to train models, but this is *doubly* impractical in a real attack scenario. First, designating a value as a mask presupposes that the attacker already knows the masking scheme in use—which intermediates carry masks, how many shares, when masks refresh—so the mask-label assumption silently inherits all of the architectural knowledge that a black-box setting explicitly disclaims. Second, even granted that knowledge, the mask values themselves are not available at run time unless invasive probing is performed—a process requiring specialized optical equipment and physical access to the chip's silicon [14,13].

2.4 Related Work

Several works form the foundation for our approach:

Multi-Task CNN (Marquet and Oswald, 2024). Marquet and Oswald [17] presented a model using Multi-Task Learning (MTL) that enables a network to learn all key byte tasks simultaneously using a Hard Parameter Sharing approach, with shared weights to identify general structural patterns in the input. The central disadvantage of this method for our setting is the requirement for information about the internal masking mechanism—the mask branch and the hard-coded XOR operation, which simultaneously predict all key bytes.

ResNet for SCA (Poudel and Rahimi, 2025). Poudel and Rahimi [22] showed that standard CNN networks suffer from the Vanishing Gradient problem, which causes difficulty in generalizing to datasets with variable keys and often results in near-random guessing. ResNet, with Residual Connections, provides a basis for faster convergence, improves gradient flow, and enables the model to generalize to variable keys.

GPAM (Bursztein and Invernizzi, 2024). Bursztein and Invernizzi [4] proposed a Transformer-based architecture with Attention mechanisms and Temporal Patchification to handle long-range sequential learning. This is a "White-Box" approach that requires access to masking labels and internal values, though it demonstrates the potential of attention-based approaches for SCA.

EstraNet (Hajra et al., 2024). Hajra et al. [6] introduced a shift-invariant transformer designed specifically for SCA, showing that attention mechanisms can be adapted to handle unaligned traces. Like GPAM, EstraNet is evaluated in a profiling setting that benefits from intermediate-value supervision. Berreby and Sauvage [2] also report recent benchmark results on ASCAD with efficient deep architectures, providing a useful reference point for the field's current state. Our quantitative comparison in section 6 is against the XOR multi-task baselines of Marquet and Oswald [17], which are the closest published architectures to our black-box setting.

3 Leakage Analysis of ASCAD_r

Before designing our architecture, we conducted a signal-to-noise ratio (SNR) analysis of the ASCAD_r dataset to characterize its leakage profile and inform architectural decisions.

3.1 First-Order SNR

For a time point t and target value s (ranging 0–255), the first-order SNR is:

$$\mathrm{SNR}(t) = \frac{\mathrm{Var}_s(\mathbb{E}[\mathrm{trace}(t) \mid s])}{\mathbb{E}_s(\mathrm{Var}[\mathrm{trace}(t) \mid s])}, \tag{3}$$

where the numerator represents the signal (variance of the conditional means) and the denominator represents the noise (mean of the within-class variances).

Our analysis confirms that bytes 0 and 1 exhibit high first-order SNR—they are unmasked and easy to extract. In contrast, **bytes 2–15 exhibit near-zero first-order SNR**, approaching the noise floor (fig. 1). This confirms that first-order attacks on the masked bytes are doomed to failure, consistent with the masking mechanism's effectiveness and aligned with findings from Marquet and Oswald [17], who also excluded these two bytes.

3.2 Second-Order SNR

To locate the second-order leakage, we implemented a centered product technique, combining pairs of time samples t_1, t_2 to neutralize the first-order component:

$$\mathrm{cp}(t_1, t_2) = \big(\mathrm{trace}(t_1) - \mu(t_1)\big) \cdot \big(\mathrm{trace}(t_2) - \mu(t_2)\big), \tag{4}$$

where $\mu(t)$ is the mean of sample t over all traces. The SO-SNR is then computed using eq. (3) on the centered product features, with averaging windows of 200 samples.

The results indicate that second-order leakage exists for all masked bytes. Since each 250K-point trace is divided into 200 windows of 1,250 samples, the window numbers translate to sample locations:

- **Mask processing:** leakage concentrated around window 100 (sample $\approx$ 125,000).
- **Masked value processing:** leakage concentrated around window 170 (sample $\approx$ 212,500).
- **Gap:** $\approx$ 87,500 samples between the two leakage events.

This gap has a critical architectural implication: the network backbone must have a receptive field wide enough to bridge both leakage events.

Per-byte variations. Bytes 7 and 12 showed SO-SNR significantly weaker than the rest, while bytes 3 and 6 showed moderate SO-SNR. This can be attributed to physical implementation factors such as register spill, memory management, and data bus transitions. As we show in section 6, this physical profile directly predicts per-byte attack difficulty.

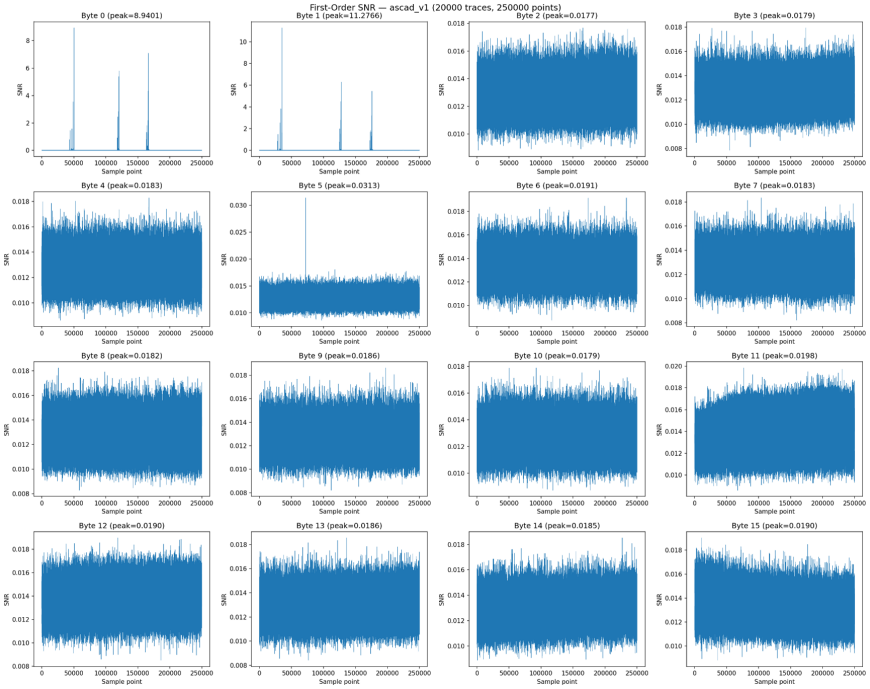


Fig. 1: First-order SNR per byte on ASCAD_r (boolean masking, 20,000 traces). Only bytes 0–1 exhibit identifiable peaks (peak SNR ≈ 4.6 and 11.3 respectively); bytes 2–15 are at the noise floor (peak SNR < 0.02), confirming the effectiveness of first-order masking.

4 Proposed Architecture

4.1 Overview

Our architecture replaces the hardcoded XorLayer with a learnable bilinear combiner while retaining the multi-task framework. The model consists of five stages:

1. **Preprocessing (PoolingCrop):** Element-wise multiplication with a learnable weight vector, followed by BatchNorm, AveragePooling1D, and AlphaDropout. This reduces the trace length while filtering irrelevant regions, creating a sufficiently wide receptive field to bridge the temporal gap identified in the SNR analysis.
2. **ResNet backbone** shared across all key bytes. The central block consists of Conv1D layers with SELU activation, BatchNorm, and AvgPool. A Skip Connection connects to the block output through an Add operation with a matched convolution, ensuring continuous gradient flow and preventing the Vanishing Gradient phenomenon. ResNet was found superior to CNN from both empirical results in earlier project stages and the literature [28,26,22].

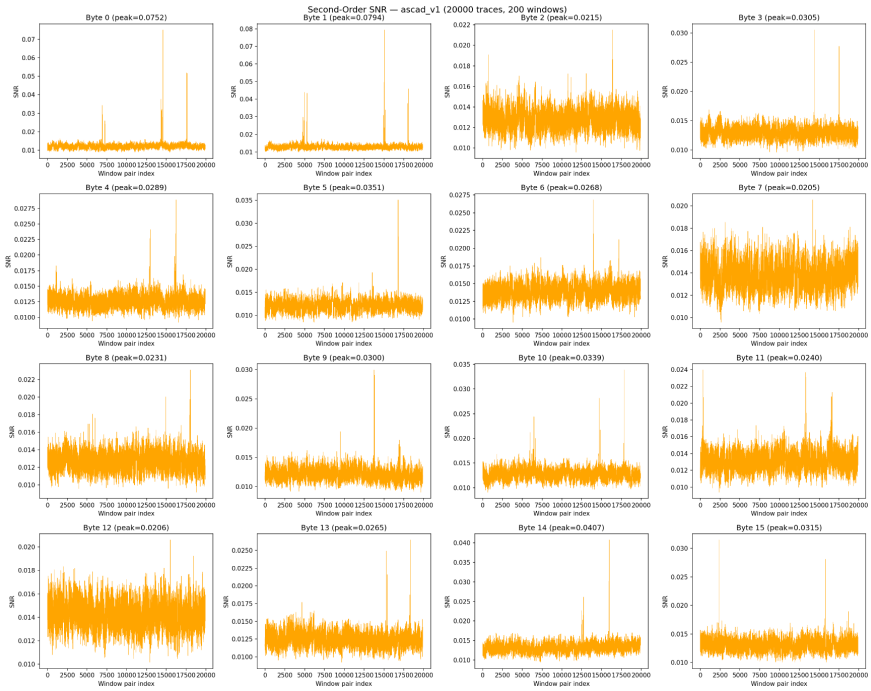


Fig. 2: Second-order SNR per byte on ASCAD_r, computed via the centered product (eq. (4)) with 200-sample windows. Per-byte SO-SNR peaks vary from ≈ 0.0008 (byte 7, weakest) to ≈ 0.075 (byte 1). The weaker bytes (3, 6, 7, 12) correlate with higher attack difficulty in section 6.

3. **Multi-task prediction heads with two components (A and B):** The backbone output is flattened and splits into two components, each containing per-byte Dense layers with SELU activation followed by SharedWeightsDenseLayers—shared weight matrices with per-byte biases enforcing AES's structural symmetry. Each component produces per-byte 256-class raw logits ($\boldsymbol{\ell}_A$, $\boldsymbol{\ell}_B$) without activation function.
4. **BilinearCombinerLayer** combining the two branches' logits with a skip connection from Component A.
5. **Softmax + Categorical Cross-Entropy loss** on the unmasked target $s_1 = \texttt{Sbox}[p \oplus k]$. No mask labels are used at any point.

4.2 Two-Component Design

We use two components (Component 0 and Component 1). The rationale: since masking can be represented as requiring the model to develop a separate representation for the masked value and for the mask itself, the dual and indepen-

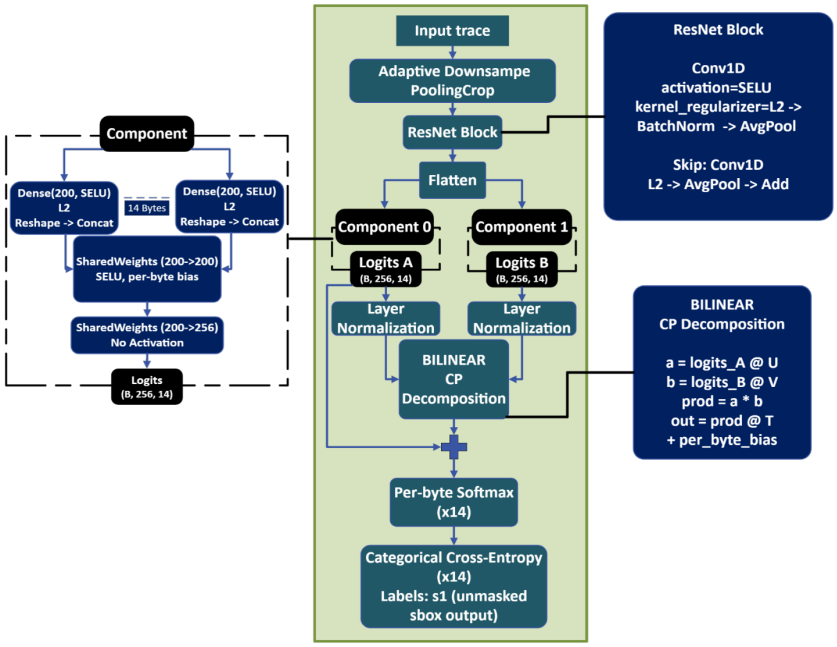


Fig. 3: Overview of the proposed masking-agnostic architecture. Raw traces pass through `PoolingCrop` preprocessing and a shared ResNet backbone, then split into two component branches (A, B) of per-byte dense layers and `SharedWeightsDense` blocks producing 256-class logits. After Layer Normalization, the bilinear CP-decomposition layer combines the two branches into the final per-byte logits, which are passed through softmax and trained with categorical cross-entropy against the unmasked SBox-output label s_1. No mask labels are used at any stage.

dent structure enables each component to specialize and learn complementary representations—to reconstruct the original value.

4.3 Learnable Bilinear Combiner

The central challenge in previous implementations was the dependency on masking-specific layers such as the hard-coded XOR operation written directly in the model code. To enable the model to naturally learn the demasking operation from the data, the outputs of both components ($\boldsymbol{\ell}_A$ and $\boldsymbol{\ell}_B$) are fed to a bilinear combination layer.

Since *first-order* masking over GF(2^8) amounts to a bilinear interaction between the mask and the masked value, CP decomposition enables efficient representation and learning of such bilinear functions directly from data [11]. The same construction extends in principle to higher-order masking: a d-share scheme corresponds to a d-linear interaction and would call for a $(d{+}1)$-dimensional CP

factorisation with d component branches feeding the combiner. We focus on the first-order (d=2) case in this work.

Instead of using a full weight tensor $\boldsymbol{W} \in \mathbb{R}^{256\times256\times256}$ (computationally expensive), the layer implements rank-R tensor decomposition:

$$W[i,j,k] \approx \sum_{r=1}^{R} U[i,r] \cdot V[j,r] \cdot T[r,k]. \tag{5}$$

The combination operation is computed as:

$$\text{output} = \big((\boldsymbol{\ell}_A \cdot \boldsymbol{U}) \odot (\boldsymbol{\ell}_B \cdot \boldsymbol{V})\big) \cdot \boldsymbol{T} + \boldsymbol{c}_b, \tag{6}$$

where:

- $\boldsymbol{U} \in \mathbb{R}^{256\times R}$ extracts features from Component A into a reduced space of size R.
- $\boldsymbol{V} \in \mathbb{R}^{256\times R}$ extracts features from Component B into the same reduced space.
- $\odot$ denotes the element-wise (Hadamard) product.
- $\boldsymbol{T} \in \mathbb{R}^{R\times256}$ projects back to the original prediction space of 256 classes.
- $\boldsymbol{c}_b$ is a per-byte bias vector.

The decomposition matrices are initialized with GlorotUniform initialization with a fixed seed. The bias vector for each byte is initialized to zeros. We use rank $R = 64$.

Coupled gradients. The element-wise product creates *direct mutual dependency in gradients*. This dependency forces Component 0 and Component 1 to specialize and learn complementary representations—at minimum, to reconstruct the original value. This makes the layer masking-agnostic, eliminating the need for hardcoded mathematical operations.

4.4 The Gradient Dead Zone

As noted during the development phases (Phase 3), the bilinear combination layer must be fed from raw logits, not from softmax probabilities.

An early attempt to implement the bilinear combination on softmax outputs led to gradient vanishing: since both branches' softmax outputs are on the order of $O(1/256)$ at initialization, the chain rule for the element-wise product gives:

$$\frac{\partial(a \cdot b)}{\partial\theta} = b \cdot \frac{\partial a}{\partial\theta} + a \cdot \frac{\partial b}{\partial\theta}, \tag{7}$$

where both a and b are $O(1/256)$, causing gradients to collapse to $O(1/256^2) \approx 1.5\times10^{-5}$. The values remain approximately identical for all 200 epochs, leaving the model in a state of random guessing.

LayerNorm is applied before the bilinear operation to prevent this gradient collapse and ensure numerical stability. Operating on logits instead of softmax probabilities maintains gradient magnitudes at $O(1)$.

4.5 Skip Connection

Before there is meaningful learning in the combination layer, adding the logits output directly from the backbone ensures the model receives useful gradients from the very first epoch:

$$\text{output} = \text{BilinearCombiner}(\boldsymbol{\ell}_A, \boldsymbol{\ell}_B) + \boldsymbol{\ell}_A. \tag{8}$$

4.6 Regularization

To stabilize training and ensure the model learns true patterns rather than memorizing training data:

- **L2 regularization** on convolution and Dense layer weights prevents filters from developing large weights, which can cause the network to rely only on the skip connection channel rather than using the bilinear combiner.
- **Gaussian noise injection:** Adding noise to the normalized traces during training forces the network to learn general patterns and not fit to specific patterns in the training traces. Trace augmentation has been a standard SCA regularizer since Cagli et al. [5], and Kim et al. [9] specifically established Gaussian noise injection as an effective regularizer for profiled DL-SCA.

4.7 Training

The model is trained using labels of SBox output only—without masking labels, and without relying on intermediate encryption data. The final output is computed through softmax layers for each byte, and the model is trained under a Categorical Cross-Entropy loss function. The model does not need to know the encryption structure or its internal values.

5 Experimental Setup

5.1 Dataset

We evaluate on ASCAD_r, a variable-key Boolean-masked AES dataset released as part of the ASCAD database by Benadjila et al. [1].

Origin and masking scheme. ASCAD_r contains side-channel traces of a software AES-128 implementation running on an 8-bit ATmega8515 microcontroller, protected by a two-share Boolean masking scheme that precomputes a masked S-Box table `SubBytes`* prior to encryption. The leakiest intermediate variables, as documented in the original ASCAD analysis [1], are the masked SubBytes input $t_i \oplus r_{\text{in}}$, the masked output $s_i \oplus r_i$, and the two masks themselves: r_{in}, the input mask, and r_i, the state mask. None of these mask values are used during training in our setup.

Splits. ASCAD_r ships with a variable-key profiling group, a variable-key test group, and a fixed-key attack group. Each trace is 250,000 samples at int8 resolution, paired with the corresponding 16-byte plaintext and key as well as 18 mask bytes (16 S-Box masks and two round-input masks). Following the convention of Marquet and Oswald [17], we use $N_{\mathrm{t}} = 50{,}000$ traces from the variable-key profiling group for training, $N_{\mathrm{v}} = 10{,}000$ traces from the variable-key test group for validation, and $N_{\mathrm{a}} = 10{,}000$ traces from the fixed-key group for the attack. The variable-key profiling group prevents the model from memorising key-specific patterns and is widely treated as a harder benchmark than the fixed-key ASCAD_v1 dataset [22].

Bytes attacked. We attack key bytes 2–15 (14 bytes); bytes 0 and 1 are unmasked in the ASCAD reference implementation and exhibit clear first-order leakage (fig. 1), so they are excluded from the masked-attack evaluation, in line with Marquet and Oswald [17].

Raw traces. As in [17], no points-of-interest selection or pre-processing is performed: the model consumes the full 250,000-sample trace as input. The best previous result reported in this raw-trace setting, Perin et al. [20], reaches single-trace success on some key bytes and requires up to three traces for the most resilient ones. Our results in section 6 fall in a comparable 2–4 trace range while operating strictly black-box—no architectural prior on the masking scheme is assumed and no mask labels are used. Where we improve cleanly is over the XOR-supervised multi-task baseline of Marquet and Oswald [17], which needs 3–14 traces for full recovery despite being told that the masking scheme is XOR.

5.2 Training Configuration

Table 1 lists the hyperparameters of the final model. Values were selected on the validation split; training is performed under a single fixed random seed.

5.3 Baselines

Primary baseline — Marquet and Oswald (XOR-supervised MTL). We compare directly against the multi-task XOR model of Marquet and Oswald [17], reproduced with the same backbone and training configuration. This baseline embeds the masking scheme into the architecture via a dedicated XorLayer and a mask branch and is therefore architectural-knowledge-dependent in the sense of section 1.

Reference point — Perin et al. (NOPOI search). The strongest result reported in our regime (no mask labels, no POI selection, raw traces) is Perin et al. [20], who obtain the per-byte trace counts (1–3 on ASCAD_r) through a large random hyperparameter search: 500 models per architecture–scenario–leakage combination over a search space of $> 10^9$ configurations, evaluated using Perceived Information. This is the result we treat as the practical state of the art in the black-box

Table 1: Hyperparameters of the final masking-agnostic model on ASCAD_r.

Component	**Parameter**	**Value**
Preprocessing	Input trace length	250 000
	Adaptive downsample target	25 000
ResNet backbone	Convolution blocks	1
	Filters per block	4
	Kernel size	34
	Strides	17
	Pooling size	2
	Activation / init	SELU / GlorotUniform
Prediction heads	Dense units per branch	200
	Non-shared dense blocks	1
	Shared dense blocks	1
Bilinear combiner	Components (A, B)	2
	CP-decomposition rank R	64
	Skip connection from A	enabled
Regularization	L2 penalty (Conv + Dense)	1×10^{-4}
	Gaussian noise std σ	0.1
Training	Optimizer	Adam
	Learning rate	1×10^{-3} (constant)
	Batch size	250
	Epochs	200
	Training traces	50 000 (subset of ASCAD_r)
	Loss	categorical cross-entropy per byte

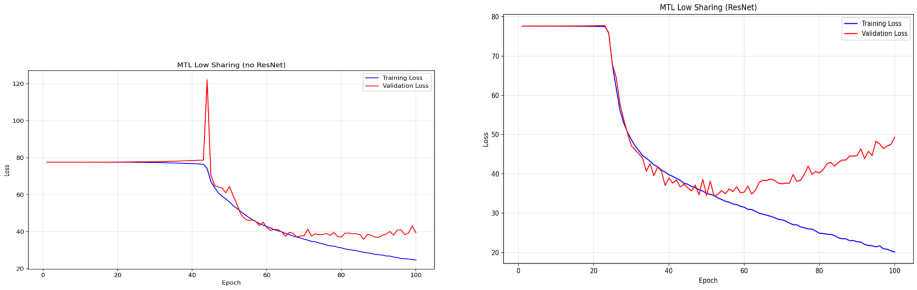
raw-trace setting; the comparison in section 6 is therefore framed against *both* Marquet (architectural prior) and Perin (search-driven generic networks).

5.4 Attack Procedure

The attack uses cumulative log-likelihood. For each candidate key byte k and N attack traces:

$$s_i = \texttt{Sbox}[p_i \oplus k], \qquad \text{Score}(k) = \sum_{i=1}^{N} \log P_{\text{model}}(s_i(k) \mid t_i). \tag{9}$$

The chosen key is $k^* = \arg\max_k \text{Score}(k)$. All 256 candidates are sorted by score in descending order. The Guessing Entropy (GE) [25] is the position of the correct key in this sorted list; GE = 1 (i.e., rank 1) indicates perfect recovery.



(a) CNN backbone: plateau lasts ≈ 40 epochs before a sharp transition, with a large validation spike around epoch 44.

(b) ResNet backbone: plateau breaks at ≈ 22 epochs and convergence is monotonic; validation diverges only past epoch 50.

Fig. 4: Training (blue) and validation (red) loss versus epoch for the MTL low-sharing baseline with (a) CNN and (b) ResNet backbones. ResNet breaks the initial plateau roughly twice as fast and converges to a lower training loss, motivating the choice of residual blocks.

6 Results and Analysis

6.1 CNN vs ResNet Backbone

In the first stage of the project, we compared a standard CNN architecture against ResNet in the Multi-Task Learning configuration from Marquet and Oswald [17].

The ResNet backbone broke through the initial learning plateau after ∼20 epochs (compared to ∼40 epochs for CNN), with exponential drops in both training and validation loss. ResNet converged significantly faster and improved the model's generalization ability. The transition to ResNet also reduced the number of traces required for key extraction in certain experiments.

6.2 Development Phases

The development of the black-box model proceeded through four phases, each addressing a specific failure mode:

Phase 1 — Direct prediction. A single prediction branch without any combination layer. The model attempted to directly predict the unmasked SBox output from masked traces. Result: complete failure—a single branch cannot perform the separation between value and mask.

Phase 2 — MLP combiner. Two branches with softmax outputs, concatenated and fed to an MLP. The assumption was that the branches would take on complementary roles. Result: failure—chaining probabilities through softmax provided independent (uncoupled) gradients; without mutual dependency, the branches

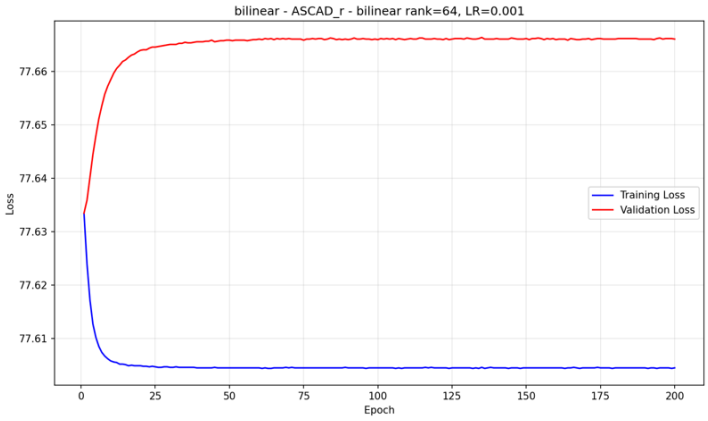


Fig. 5: Phase 3 —- bilinear combiner applied to softmax probabilities. Both losses remain flat at ≈ 77.6 (the random-guessing baseline for 14 outputs of 256 classes, $14 \times \ln 256$) over all 200 epochs. Gradients collapse to $O(1/256^2)$ and no learning occurs.

had no opportunity to specialize. The model exhibited significant overfitting, memorizing training data without generalization.

Phase 3 — Bilinear on softmax outputs. The MLP was replaced with the bilinear combiner based on CP decomposition, operating on softmax outputs. Result: failure—the gradient dead zone ($O(1/256^2)$ gradient magnitude) prevented any learning. Values remained approximately identical for all 200 epochs (fig. 5).

Skip connection alone is insufficient. Introducing the skip connection on raw logits (section 4.5) without L2 regularization or noise injection produces a different failure mode (fig. 6): the training loss now descends after a long plateau, but the validation loss diverges, indicating the model memorizes the training set through the unregularized skip path rather than exploiting the bilinear combiner.

Phase 4 — Bilinear on raw logits + skip + regularization. The final architecture, operating on raw logits with rank $R = 64$, combined with skip connection, L2 regularization, and Gaussian noise injection. This resolved the convergence problems (fig. 7).

Table 2 summarizes the progression: each row toggles one component relative to the previous one and reports the resulting failure (or success) mode.

6.3 Learning Analysis

Tracking the learning curve reveals the influence of the model's components over time:

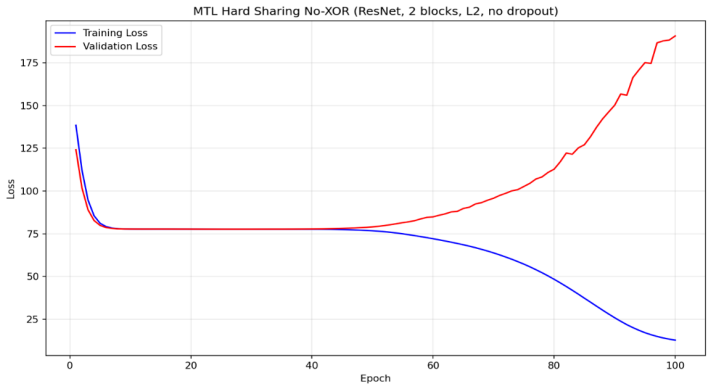


Fig. 6: Skip connection alone (no L2, no noise). After a $\sim$ 50-epoch plateau, the training loss descends rapidly but the validation loss diverges from ≈ 77 to ≈ 190, demonstrating that without regularization the model overfits through the skip path rather than learning the bilinear combiner.

Table 2: Ablation across the development phases. Each row enables one architectural component (combiner type, skip connection, regularization) relative to the previous row. Only the full configuration converges.

Phase	**Combiner**	**LayerNorm**	**Skip**	**L2 + Noise**	**Outcome**
1	none (direct prediction)	—	—	—	complete failure: single branch cannot demask
2	MLP on softmax	—	—	—	overfit; gradients are uncoupled
3	bilinear on softmax	—	—	—	dead zone ($O(1/256^2)$ gradients)
–	bilinear on raw logits	✓	✓	—	training descends, validation diverges
4	bilinear on raw logits	✓	✓	✓	**converges; GE** $= 0$ **at 2–4 traces**

– **Epochs 1–50:** The model predicts randomly. The ResNet backbone is not yet extracting features. The skip connection allows gradients to begin flowing from the very first epoch.

– **Epochs 75–100:** The model begins to successfully learn the inverse demasking operation, achieving training accuracy of 77.3% and validation accuracy of 41.7%. The training–validation gap on cross-entropy is expected: prior work has shown that accuracy and cross-entropy are only loosely correlated with key-recovery performance for SCA [18,21], and we therefore rely on GE on the attack split (section 6) as the operational metric.

– **Epochs 150–200:** The model converges, demonstrating generalization ability without overfitting.

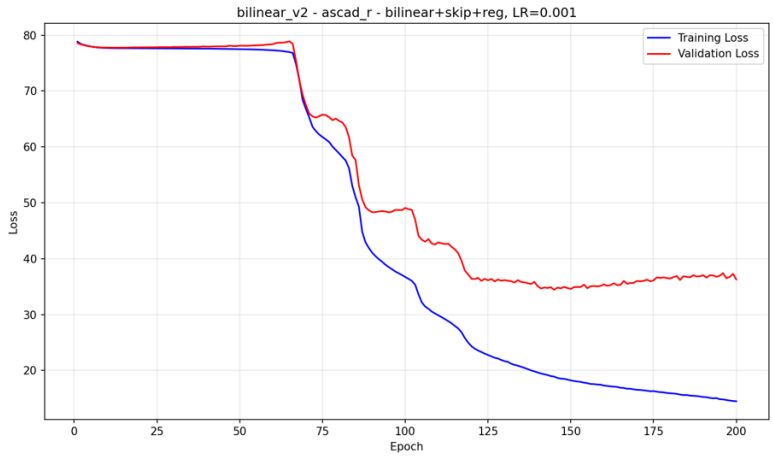


Fig. 7: Phase 4 —- final architecture with bilinear combiner on raw logits, skip connection, L2 regularization, and Gaussian noise injection. After a $\sim$ 60-epoch plateau the model breaks through, with both training and validation losses descending; validation stabilizes around 35–37, confirming generalization without overfitting.

6.4 Attack Results

The chosen model's key extraction performance was measured by Guessing Entropy (GE) as a function of the number of attack traces. The following conclusions emerge:

- Against the XOR-supervised baseline of Marquet and Oswald [17], the bilinear-combiner model ranks the correct key higher for every number of traces used in the attack, despite the baseline having hardcoded knowledge of the masking scheme.
- Even given a single trace, the model places the correct key byte in the top position in a significant percentage of cases (31.7%–67.2% depending on the byte, see table 4).
- When accumulating probabilities over traces, the model needs only **2**–**4** traces for complete key recovery, compared to 3–14 traces for the XOR-supervised baseline. Both numbers fall within the trace range reported by the search-driven black-box baseline of Perin et al. [20] (1–3 traces per byte on ASCAD_r), confirming that the structural-prior approach is competitive with 500-trial random search.
- Performance is matched without the architecture being told that the masking scheme is XOR: the bilinear combiner discovers the demasking operation from data.

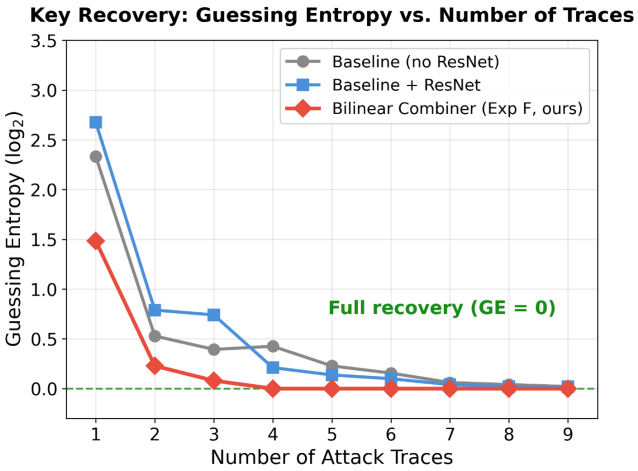


Fig. 8: Guessing entropy (log scale) versus number of accumulated attack traces, averaged across the 14 masked bytes. The bilinear combiner (red) reaches GE = 0 (perfect recovery) by trace 4, compared to trace 6 for the XOR baseline with ResNet (blue) and trace 7 for the original XOR baseline (gray). Our model wins despite having to *learn* the demasking operation rather than being told it is XOR.

6.5 Per-Byte Difficulty and Correlation with SO-SNR

Table 3 reports single-trace top-1 accuracy per byte for all three models. The bilinear combiner outperforms both XOR baselines across every byte (mean 38.1% vs. 27.8% for XOR low-sharing + ResNet and 20.7% for XOR hard-sharing + ResNet), and the within-row ranking of difficulty is preserved across architectures: bytes that are hard for one model are hard for all of them. This consistency is the key observation behind our claim that difficulty is an inherent property of the physical leakage, not of the model.

Table 4 provides a more granular view of the same attack from a second, separately-seeded training of the same configuration, evaluating the rank of the correct key byte on each of the 10,000 attack traces. For every byte the median rank is 1–3 and the correct key falls in the top 3 on at least 59% of single traces. On the easiest bytes (4, 5, 9, 11, 14) the model places the correct key first on roughly two-thirds of single traces and within the top 3 on more than 90%. On the hardest byte (3, which has only moderate SO-SNR), the correct key is still ranked first 31.7% of the time on a single trace and within the top 3 59.6% of the time.

Examination of SO-SNR values explains the within-row pattern:

- Bytes 7 and 12 hold the weakest SO-SNR values; difficulty in their extraction most likely stems from the weakness of the physical signal.
- Bytes 3 and 6 have moderate SO-SNR.

Table 3: Per-byte single-trace top-1 accuracy (%) on the attack split. Random-guessing baseline is $1/256 \approx 0.39\%$. The bilinear combiner wins every byte and improves the mean by +10.3 percentage points over the strongest XOR baseline. SO-SNR category from section 3: H = high, M = moderate, L = low.

Byte	2	3	4	5	6	7	8	9	10	11	12	13	14	15	**Mean**
SO-SNR	H	M	H	H	M	L	H	H	H	H	L	H	H	H	—
XOR Hard + ResNet	18.1	16.8	21.0	22.6	17.0	14.9	20.3	17.8	22.6	17.9	21.0	20.3	22.7	36.5	20.7
XOR Low + ResNet	27.5	25.1	29.5	30.1	24.9	21.8	29.0	25.4	30.8	27.5	25.7	29.6	27.0	35.3	27.8
Bilinear (ours)	**39.1**	**23.2**	**42.9**	**52.8**	**40.2**	**32.6**	**30.7**	**43.8**	**29.5**	**48.9**	**22.8**	**34.0**	**52.7**	**40.6**	**38.1**

Table 4: Detailed per-byte single-trace rank statistics for our final model on the full 10,000-trace attack split (run with seed independent of table 3). "Top-k" columns report the percentage of single traces on which the correct key byte ranks within the top k out of 256 candidates.

Byte	**Mean**	**Median**	**Top-1%**	**Top-3%**	**Top-5%**	**Top-10%**	**Top-20%**
2	2.3	1	54.6	85.4	93.6	98.4	99.7
3	5.0	3	31.7	59.6	73.9	89.2	96.4
4	1.8	1	60.8	90.4	96.9	99.6	100.0
5	1.8	1	60.6	91.0	97.2	99.8	100.0
6	1.9	1	59.4	89.6	96.2	99.4	99.9
7	3.1	2	44.2	76.6	87.5	96.2	99.1
8	3.2	2	43.3	74.4	86.5	95.7	99.1
9	1.6	1	67.2	94.4	98.8	99.9	100.0
10	3.2	2	47.5	77.3	87.5	95.1	98.2
11	1.7	1	66.0	92.8	97.6	99.7	100.0
12	3.9	2	39.9	69.8	82.2	92.8	97.9
13	3.5	2	39.8	70.7	83.0	94.1	99.0
14	1.7	1	66.2	93.6	98.0	99.7	100.0
15	2.6	2	48.3	79.0	89.8	97.6	99.8
Mean	**2.7**	—	**52.1**	**82.5**	**90.6**	**96.9**	**99.2**

– The attack difficulty trend is consistent across *all tested architectures* (including the hardcoded XOR baseline), indicating that the central performance barrier is influenced by the physics of the chip, the processing order in the software implementation, and the signal distribution in the trace.
– The difficulty in extracting certain bytes does not stem from a limitation of the deep learning model, but largely reflects an inherent limitation of the information level present in the physical data itself.

7 Discussion

7.1 Architectural Insights

From the development phases and empirical findings, several insights emerge for training models to extract information from masked systems:

The network backbone as bottleneck. The second-order leakage indeed exists in the data, and therefore the central challenge is to correctly locate and route

the features for each byte. Usage of PoolingCrop along with a ResNet-based architecture was essential to create a sufficiently wide receptive field to bridge the temporal gap between the mask and masked value leakage events.

CP decomposition's representational capability. It was proven that tensor decomposition of sufficient rank is capable of representing and learning bilinear functions over $\mathrm{GF}(2^8)$, and can successfully learn these operations directly from the data, eliminating the need for prior knowledge about the masking structure.

Gradient flow: logits with normalization vs. softmax probabilities. A key insight is that the combination layer must operate on raw logits with normalization. Attempts to apply the combination on softmax probabilities maintained gradient attenuation at $O(1/256^2)$ and caused model failure.

Balance between learning and memorization. The model's success relies on synergy between multiple components. The skip connection provides useful gradients to the backbone from the first epoch. Without L2 regularization and Gaussian noise injection, the model tends to memorize data and rely only on the skip connection channel. The regularization forces the network to use the bilinear combiner to find a generalized solution.

Attack difficulty as inherent data property. The consistent difficulty in extracting certain bytes across all tested architectures points to the physics of the implementation—an inherent limitation of the information level present in the physical data—as the central performance barrier, not a limitation of the deep learning model.

Efficiency of explicit structure vs. hyperparameter search. In the black-box raw-trace regime our trace counts (2–4) are competitive with the strongest published result of Perin et al. [20] (1–3 on ASCAD_r), reached by random-searching 500 MLP/CNN configurations per scenario over a 10^9-option search space. The bilinear combiner reaches the same range through a 4-phase manual ablation of a single architecture family, suggesting that an explicit structural inductive bias—matching the algebraic class of the underlying demasking operation—can substitute for brute-force architecture search. The benefit is twofold. First, the search cost collapses from $O(10^3)$ random trials to a handful of ablations. Second, the resulting architecture is interpretable: the CP-decomposition structure makes clear *what* the network is learning and exposes a clean route to higher-order generalisation (section 4.3), whereas a generic MLP or CNN gives no comparable structural handle. We view this as a representational, not numerical, contribution: comparable performance from a small, principled search rather than a large, undirected one.

7.2 Scope and Limitations

First-order Boolean masking focus. Our experiments focus on first-order Boolean masking (ASCAD_r), which is the most widely deployed masking scheme. The

architecture is agnostic by design within this class—the BilinearCombinerLayer makes no assumptions about which bilinear operation to learn—but the present two-component construction is matched to two-share masking. Higher-order (d>1) masking is d-linear rather than bilinear and would require the natural extension discussed in section 4.3: d component branches and a (d+1)-dimensional CP factorisation. We did not run this extension.

Affine masking. We also evaluated the architecture on ASCAD v2 (affine masking) under the same training pipeline, but did not achieve key recovery within our compute budget. Affine masking introduces multiplication in GF(2^8) in addition to XOR; we conjecture that the rank-64 CP decomposition, while sufficient to discover XOR demasking, may underrepresent the larger algebraic structure required for affine demasking, and that a higher rank, additional component branches, or curriculum scheduling between boolean and affine targets are needed. A more thorough investigation is left to future work.

Single seed evaluation. All reported numbers use a single fixed random seed (`seed=42`). We did not run a multi-seed study with mean and standard deviation, which means our reported GE numbers should be read as representative of one full training run rather than as a tight confidence interval. The qualitative ordering of phases and models is robust to this choice (each phase changes the behavior categorically, not by a small margin), but exact trace counts may vary by a small amount across seeds.

Dataset and implementation scope. ASCAD_r is collected from a single 8-bit ATmega8515 microcontroller running a specific software AES-128 implementation. We do not claim that the learned bilinear combiner transfers across devices, supply voltages, compilers, or implementation styles without retraining. Generalization across implementations—in particular, whether a single profiling model can attack multiple devices or whether per-target retraining is required—is an open question that demands experiments outside the scope of this paper.

Computational cost. Training the final configuration requires approximately 48 hours on a single NVIDIA A6000 GPU for 200 epochs over 50,000 training traces. This is dominated by the long input length (250,000 samples) and the 14-byte multi-task output; profiling cost is amortized across all subsequent attack runs but is not negligible.

8 Conclusion

This project develops a side-channel attack model, based on deep learning, that operates as a "black box"—without any prior knowledge about the encryption implementation and without reliance on internal intermediate data. Combining a ResNet backbone, Multi-Task Learning, and a learnable bilinear combination layer based on CP decomposition, the model replaces hand-coded demasking

operations with an algebraic structure discovered from data. On ASCAD_r it recovers all 14 masked key bytes with GE = 2^0 in 2–4 attack traces, within the trace range of the strongest published black-box result [20] and clearly better than the XOR-supervised baseline of Marquet and Oswald [17] (3–14 traces).

The contribution is representational, not numerical: where prior black-box work obtained comparable performance through random search over generic MLP/CNN configurations [20], we obtain it through a 4-phase manual ablation of an architecture whose inductive bias matches the algebraic class of the demasking operation. The model learns to perform XOR demasking without being told that the masking scheme is XOR, the construction generalises naturally to higher-order masking via $(d{+}1)$-dimensional CP decomposition, and the analysis surfaces a gradient dead zone that arises whenever bilinear operations are applied to softmax probabilities—a finding that may be useful beyond this paper. A signal-to-noise ratio analysis confirms that per-byte difficulty variations correlate with the physical leakage profile, identifying the physics of the implementation as the remaining performance bottleneck.

Future work will focus on:

1. Replacing the ResNet backbone with Transformer architectures and Attention mechanisms, which may enable higher quality feature extraction from longer and more complex traces.
2. Adaptation to more complex masking schemes such as Affine Masking and higher-order Boolean masking. Higher-order masking is d-linear in the share count and maps onto a $(d{+}1)$-dimensional CP decomposition with d component branches feeding the combiner—the same construction we use here, instantiated at higher order. Affine masking is more delicate: the multiplicative share lives in $\mathrm{GF}(2^8)^\times$, which may also require different model parameters, CP rank, or curriculum scheduling.
3. Investigating sensitivity of the per-byte difficulty profile to compiler optimization effects—examining whether recompiling the AES code on the same microcontroller under different compilation settings (e.g., different register allocation, alternative memory access patterns) changes which bytes are easy or difficult to extract. Results from such an experiment would provide empirical proof that attack difficulty is an artifact of the software implementation at the machine level, and could guide the development of more secure compilation methods against side-channel attacks.
4. Exploring SCA-specific loss functions [7] as alternatives to categorical cross-entropy. Since cross-entropy is misaligned with the actual key-recovery objective, ranking-based and guessing-entropy-aware losses may further reduce the number of attack traces required.

References

1. Benadjila, R., Prouff, E., Strullu, R., Cagli, E., Dumas, C.: Deep learning for side-channel analysis and introduction to ASCAD database. Journal of Cryptographic Engineering **10**(2), 163–188 (2020). https://doi.org/10.1007/s13389-019-00220-8

2. Berreby, Y.E., Sauvage, L.: Investigating efficient deep learning architectures for side-channel attacks on AES. arXiv preprint arXiv:2309.13170 (2023). https://doi.org/10.48550/arXiv.2309.13170, https://arxiv.org/abs/2309.13170
3. Brier, E., Clavier, C., Olivier, F.: Correlation power analysis with a leakage model. In: Cryptographic Hardware and Embedded Systems – CHES 2004. LNCS, vol. 3156, pp. 16–29. Springer (2004). https://doi.org/10.1007/978-3-540-28632-5_2
4. Bursztein, E., Invernizzi, L., Král, K., Moghimi, D., Picod, J.M., Zhang, M.: Generalized power attacks against crypto hardware using long-range deep learning. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(3), 472–499 (2024). https://doi.org/10.46586/tches.v2024.i3.472-499
5. Cagli, E., Dumas, C., Prouff, E.: Convolutional neural networks with data augmentation against jitter-based countermeasures — profiling attacks without pre-processing. In: Cryptographic Hardware and Embedded Systems – CHES 2017. LNCS, vol. 10529, pp. 45–68. Springer (2017). https://doi.org/10.1007/978-3-319-66787-4_3
6. Hajra, S., Chowdhury, S., Mukhopadhyay, D.: EstraNet: An efficient shift-invariant transformer network for side-channel analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(1), 336–374 (2024). https://doi.org/10.46586/tches.v2024.i1.336-374
7. Kerkhof, M., Wu, L., Perin, G., Picek, S.: No (good) loss no gain: systematic evaluation of loss functions in deep learning-based side-channel analysis. Journal of Cryptographic Engineering **13**(3), 311–324 (2023). https://doi.org/10.1007/s13389-023-00320-6
8. Khalaj Monfared, S., Mitard, K., Cannon, A., Forte, D., Tajik, S.: LaserEscape: Detecting and mitigating optical probing attacks. In: IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24). ACM (2024). https://doi.org/10.1145/3676536.3676822
9. Kim, J., Picek, S., Heuser, A., Bhasin, S., Hanjalic, A.: Make some noise: Unleashing the power of convolutional neural networks for profiled side-channel analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems **2019**(3), 148–179 (2019). https://doi.org/10.13154/tches.v2019.i3.148-179
10. Kocher, P., Jaffe, J., Jun, B.: Differential power analysis. In: Advances in Cryptology — CRYPTO '99. LNCS, vol. 1666, pp. 388–397. Springer-Verlag (1999). https://doi.org/10.1007/3-540-48405-1_25
11. Kolda, T.G., Bader, B.W.: Tensor decompositions and applications. SIAM Review **51**(3), 455–500 (2009). https://doi.org/10.1137/07070111X
12. Krachenfels, T., Ganji, F., Moradi, A., Tajik, S., Seifert, J.P.: Real-world snapshots vs. theory: Questioning the t-probing security model. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1955–1971. IEEE (2021)
13. Krachenfels, T., Kiyan, T., Tajik, S., Seifert, J.P.: Automatic extraction of secrets from the transistor jungle using Laser-Assisted Side-Channel attacks. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 627–644 (2021)
14. Lohrke, H., Tajik, S., Boit, C., Seifert, J.P.: No place to hide: Contactless probing of secret data on FPGAs. In: Cryptographic Hardware and Embedded Systems – CHES 2016. pp. 147–167. Springer (2016)
15. Maghrebi, H., Portigliatti, T., Prouff, E.: Breaking cryptographic implementations using deep learning techniques. In: Security, Privacy, and Applied Cryptography Engineering (SPACE 2016). LNCS, vol. 10076, pp. 3–26. Springer (2016). https://doi.org/10.1007/978-3-319-49445-6_1

16. Mangard, S., Oswald, E., Popp, T.: Power Analysis Attacks: Revealing the Secrets of Smart Cards. Advances in Information Security, Springer (2007)
17. Marquet, T., Oswald, E.: Exploring multi-task learning in the context of masked AES implementations. In: Constructive Side-Channel Analysis and Secure Design (COSADE 2024). pp. 93–112. LNCS, Springer (2024). https://doi.org/10.1007/978-3-031-57543-3_6
18. Masure, L., Dumas, C., Prouff, E.: A comprehensive study of deep learning for side-channel analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems **2020**(1), 348–375 (2020). https://doi.org/10.13154/tches.v2020.i1.348-375
19. National Institute of Standards and Technology: Advanced encryption standard (AES). Tech. Rep. FIPS 197, U.S. Department of Commerce (2001). https://doi.org/10.6028/NIST.FIPS.197
20. Perin, G., Wu, L., Picek, S.: Exploring feature selection scenarios for deep learning-based side-channel analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems **2022**(4), 828–861 (2022). https://doi.org/10.46586/tches.v2022.i4.828-861
21. Picek, S., Heuser, A., Jovic, A., Bhasin, S., Regazzoni, F.: The curse of class imbalance and conflicting metrics with machine learning for side-channel evaluations. IACR Transactions on Cryptographic Hardware and Embedded Systems **2019**(1), 209–237 (2019). https://doi.org/10.13154/tches.v2019.i1.209-237
22. Poudel, M., Rahimi, N.: Machine learning-based AES key recovery via side-channel analysis on the ASCAD dataset. arXiv preprint arXiv:2508.11817 (2025). https://doi.org/10.48550/arXiv.2508.11817, https://arxiv.org/abs/2508.11817
23. Prouff, E., Rivain, M., Bevan, R.: Statistical analysis of second order differential power analysis. IEEE Transactions on Computers **58**(6), 799–811 (2009). https://doi.org/10.1109/TC.2009.15
24. Staib, M., Moradi, A.: Deep learning side-channel collision attack. IACR Transactions on Cryptographic Hardware and Embedded Systems **2023**(3), 422–444 (2023). https://doi.org/10.46586/tches.v2023.i3.422-444
25. Standaert, F.X., Malkin, T.G., Yung, M.: A unified framework for the analysis of side-channel key recovery attacks. In: Advances in Cryptology – EUROCRYPT 2009. LNCS, vol. 5479, pp. 443–461. Springer (2009). https://doi.org/10.1007/978-3-642-01001-9_26
26. Wouters, L., Arribas, V., Gierlichs, B., Preneel, B.: Revisiting a methodology for efficient CNN architectures in profiling attacks. IACR Transactions on Cryptographic Hardware and Embedded Systems **2020**(3), 147–168 (2020). https://doi.org/10.13154/tches.v2020.i3.147-168
27. Wu, L., Perin, G., Picek, S.: Weakly profiling side-channel analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(3), 707–730 (2024). https://doi.org/10.46586/tches.v2024.i3.707-730
28. Zaid, G., Bossuet, L., Habrard, A., Venelli, A.: Methodology for efficient CNN architectures in profiling attacks. IACR Transactions on Cryptographic Hardware and Embedded Systems **2020**(1), 1–36 (2020). https://doi.org/10.13154/tches.v2020.i1.1-36